

Activity 2:

Goal to understand:

(a) Self-attention mechanism

(b) Vector embedding

(c) Feed forward Neural Network

How Transformers Work — Summary

Sources used:

1. [Codecademy — Transformer Architecture Explained With Self-Attention Mechanism](#) — full walkthrough of encoder/decoder, QKV math, feed-forward layer
2. [IBM — What Are Word Embeddings?](#) — embeddings fundamentals, Word2Vec/GloVe history
3. Academic/arXiv sources on self-attention mechanics (formal QKV notation, multi-head attention)

(a) Vector Embedding — the starting point

Neural networks only understand numbers, not text. So the first step is converting text into numbers:

1. Tokenization: input text is broken into subword units called tokens (e.g., "reading" → "read" + "ing").
2. Embedding: each token is mapped to a vector of real numbers — a point in a high-dimensional space (e.g., 300–4096 dimensions) — via a learned embedding table.
3. Why it works: the position and direction of a vector encodes meaning. Words used in similar contexts end up close together in this space, and relationships become geometric — the classic example is *king – man + woman ≈ queen*.
4. Static vs. contextual limitation: a plain embedding gives "watch" the same vector whether it means a timepiece or "to watch a movie." That's the problem self-attention exists to solve — turning a *static* embedding into a *contextual* one based on surrounding words.
5. Positional encoding: since transformers process all tokens in parallel (unlike RNNs), position information ("The cat chased the dog" vs. "The dog chased the cat") is

injected separately by adding a positional encoding vector to each token embedding.

(b) Self-Attention Mechanism — the core innovation

Self-attention lets every token "look at" every other token in the sequence and decide how much to weight each one when building its own contextual meaning. Mechanically:

1. Three vectors per token: each token's embedding is projected (via learned weight matrices) into a Query (Q), Key (K), and Value (V) vector.
 - Query = "what am I looking for?"
 - Key = "what do I contain, that others might want?"
 - Value = "what information do I actually contribute?"
2. Score relevance: for token i attending to token j , compute the dot product $Q_i \cdot K_j$ — a similarity score.
3. Scale and normalize: divide by $\sqrt{d_k}$ (embedding dimension) to stabilize gradients, then apply softmax so all scores across the sequence sum to 1 (i.e., become attention "weights").
4. Weighted sum: multiply each weight by its corresponding Value vector and sum them all → this produces the new, context-aware embedding for token i .
 - In matrix form: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$
5. Multi-head attention: this whole process runs several times in parallel ("heads"), each learning to focus on a different kind of relationship (syntax, semantics, position). Outputs from all heads are combined.
6. Masking (decoder only): when generating text, a mask blocks each token from attending to future tokens, forcing left-to-right, causal generation.

The famous illustrative example: in *"The computer executes the program because it is told to,"* self-attention lets the model correctly link "it" back to "computer" rather than "program" — something static embeddings alone can't do.

(c) Feed-Forward Neural Network — the enrichment step

After the attention layer produces a context-aware vector for each token, that vector passes through a position-wise feed-forward network (FFN):

- It's a simple two-layer fully-connected network (usually expand → non-linear activation → contract back to original dimension).
- It's applied independently and identically to each token's vector (not across tokens — attention already handled cross-token mixing).
- Its job is to apply a non-linear transformation, giving the model additional capacity to learn complex patterns beyond what the (linear-ish) attention weighting alone can capture.
- After both the attention and feed-forward sub-layers, a residual connection + layer normalization ("Add & Norm") is applied — this adds the sublayer's input back to its output, which stabilizes training and keeps gradients flowing smoothly through deep stacks of these blocks.